

Student Information Management System

DBMS Mini Project Report

A MySQL-based database-driven application demonstrating the concepts covered in the supplied Database Management System Laboratory manual.

Technologies: MySQL 8.0+, PHP 8+, HTML5, CSS3, InnoDB

Project Type: Student Information Management System

Prepared as a DBMS laboratory mini project

1. Introduction

The Student Information Management System (SIMS) is a relational database application designed to demonstrate core Database Management System concepts in one coherent project. Instead of treating each SQL topic as an isolated exercise, SIMS connects students, departments, courses, enrollments, faculty and results through normalized tables and foreign-key relationships.

The database uses MySQL and the interface uses PHP with prepared statements. The project includes CRUD operations, constraints, joins, SQL functions, subqueries, views, grouping, indexing, transactions, stored routines, triggers and DCL examples.

2. Problem Statement

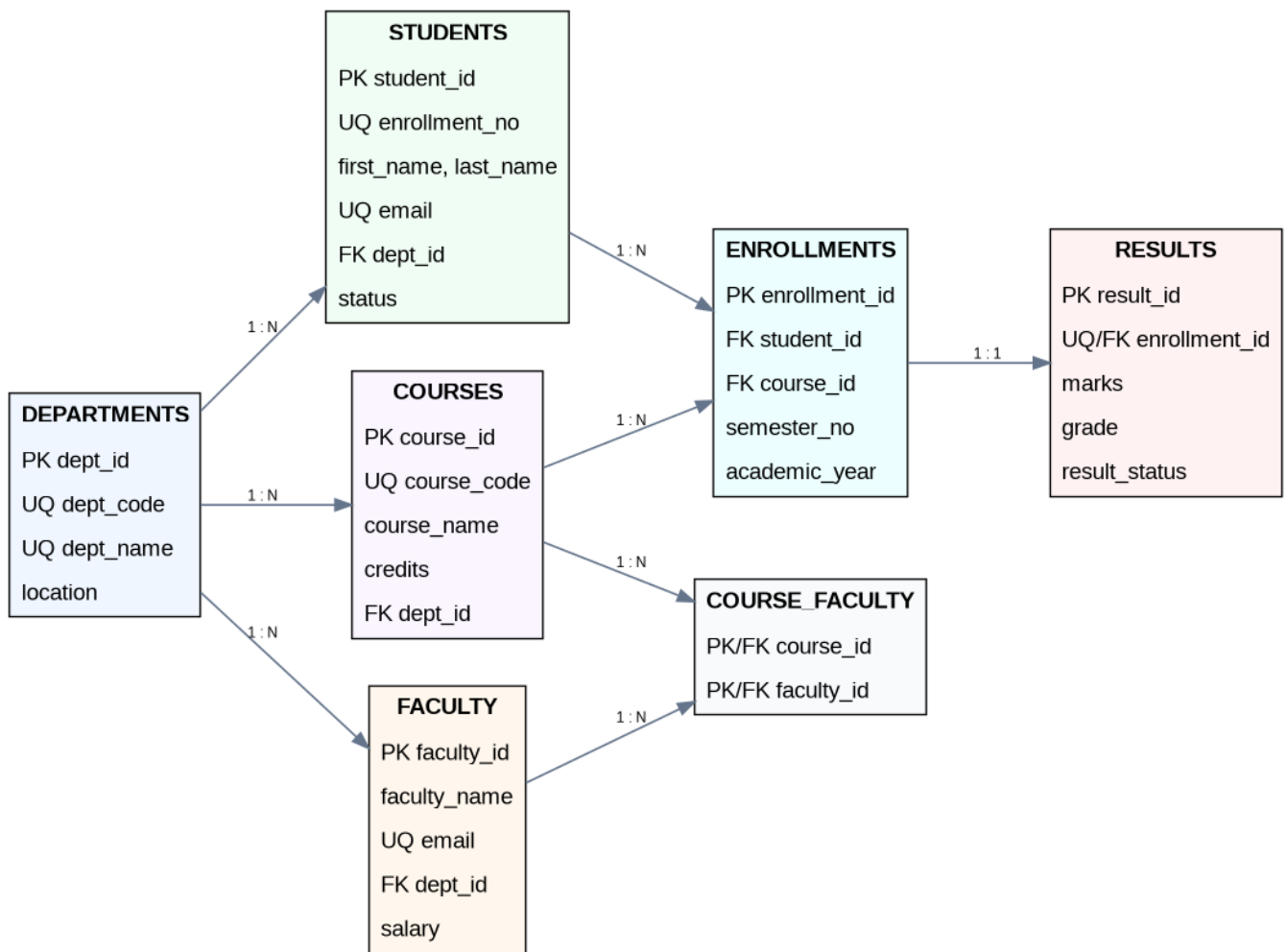
Educational institutions manage information about students, departments, courses, faculty, enrollments and examination results. If this information is stored in unrelated spreadsheets or poorly structured tables, duplication, inconsistent updates and weak data integrity can occur. The project solves this by providing a centralized relational database with integrity constraints and a simple web interface for data operations.

3. Objectives

- Design a well-structured relational schema with primary and foreign keys.
- Apply normalization concepts and reduce update anomalies.
- Provide user-friendly CRUD operations for core student records.
- Demonstrate SQL DDL, DML, functions, operators, joins, grouping, indexing, subqueries and views.
- Implement transactions, stored routines, triggers and DCL examples.
- Provide database setup, sample data, backup/recovery instructions and GitHub-ready documentation.

4. ER Diagram

The ER diagram below shows the main entities and their cardinality. Departments connect to students, faculty and courses. Students and courses form a many-to-many relationship through enrollments, while results are associated with an enrollment. Faculty and courses are connected through the course_faculty bridge table.



5. Database Schema

Table	Purpose	Key relationships
departments	Department master data	Referenced by students, faculty, courses
students	Student personal and admission data	FK to departments
faculty	Faculty details and salary	FK to departments
courses	Course master data	FK to departments
enrollments	Student-course registration	FK to students and courses
course_faculty	Course-teacher mapping	FK to courses and faculty
results	Marks, grade and result status	One-to-one with enrollments
student_audit_log	Trigger-generated audit trail	Captures student insert/update/delete

Normalization

The core design separates department, student, course and faculty facts into their own relations. The enrollments table stores the relationship between students and courses, avoiding repeating course columns inside the student table. The course_faculty table stores another many-to-many relationship. This supports 1NF through atomic values, 2NF by avoiding partial dependency in bridge records, and 3NF by keeping non-key attributes dependent on their relevant key.

6. Technologies Used

Layer	Technology
Database	MySQL 8.0+ / InnoDB
Backend	PHP 8+ / PDO
Frontend	HTML5 / CSS3
Documentation	Markdown + PDF
Version control	Git / GitHub-ready repository structure

7. SQL Queries and Features Implemented

Manual area	Implementation
DDL	CREATE DATABASE/TABLE, ALTER TABLE ADD/MODIFY/RENAME, RENAME TABLE, DROP TABLE
DML + CRUD	INSERT, UPDATE, DELETE plus web forms for students/departments/courses/enrollments/results
Functions	Number, aggregate, character, conversion and date functions
Operators	Arithmetic, logical, comparison, BETWEEN, NULL, LIKE, IN, EXISTS, ANY, ALL and set operations
Joins	INNER, LEFT, RIGHT, SELF, NATURAL, non-equi and MySQL full-outer-join emulation
GROUP / ORDER / INDEX	GROUP BY, HAVING, ORDER BY and application indexes
Subqueries / Views	Scalar, IN, correlated and EXISTS examples; two reporting views
Constraints	PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT
TCL	START TRANSACTION, SAVEPOINT, ROLLBACK, COMMIT
Backup / Recovery	mysqldump and mysql restore instructions
DCL	CREATE USER, CREATE ROLE, GRANT, REVOKE, ALTER USER, DROP USER examples
PL/SQL equivalent	MySQL stored procedure and stored function
Triggers	Result grade/status triggers and student audit triggers

Representative SQL

Join: SELECT s.enrollment_no, CONCAT(s.first_name, ' ', s.last_name), d.dept_name FROM students s JOIN departments d ON s.dept_id=d.dept_id;

Subquery: SELECT * FROM faculty WHERE salary > (SELECT AVG(salary) FROM faculty);

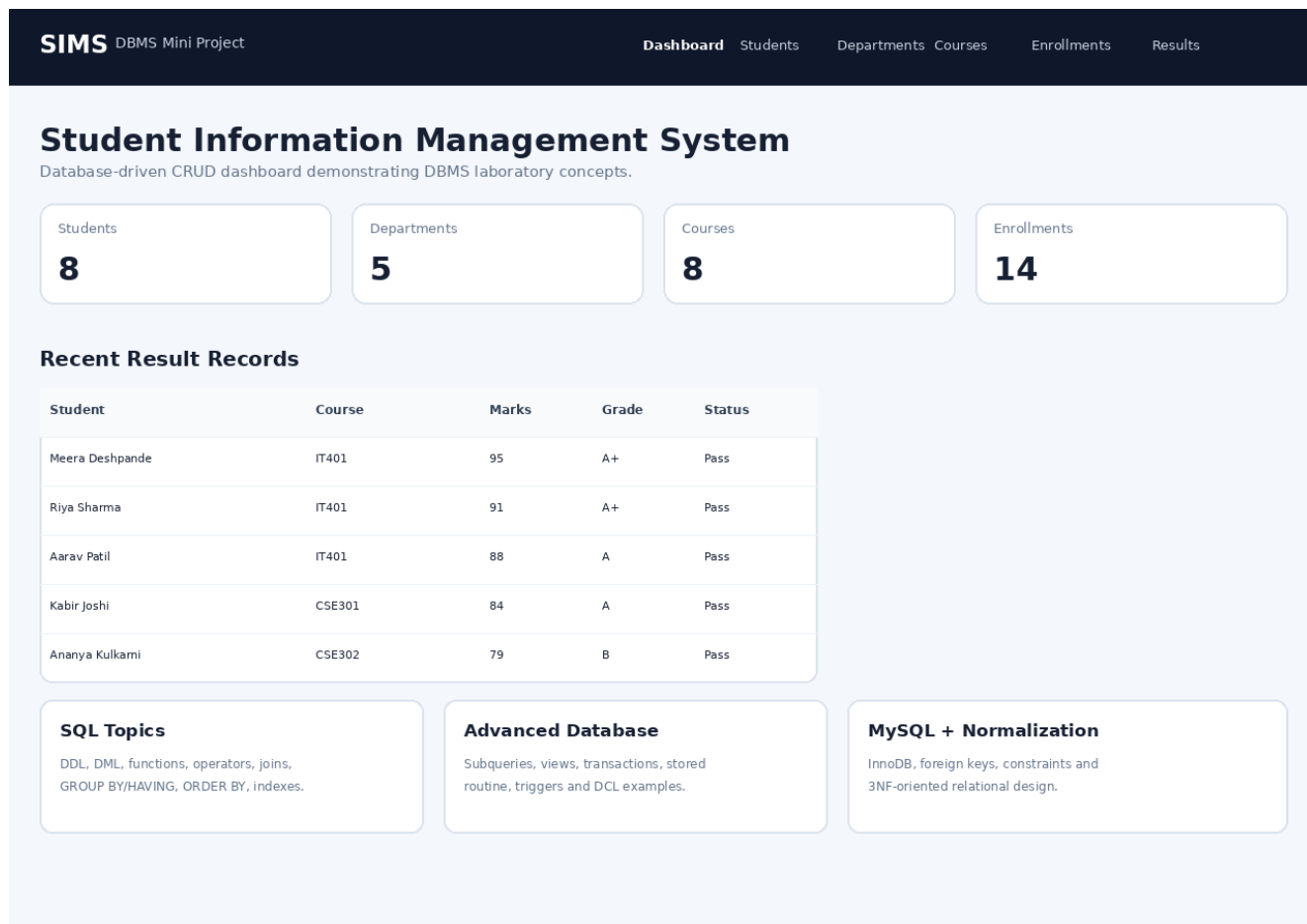
View: CREATE VIEW v_student_results AS SELECT ... FROM students JOIN enrollments JOIN courses JOIN results;

Transaction: START TRANSACTION; UPDATE ...; SAVEPOINT ...; ROLLBACK TO SAVEPOINT ...; COMMIT;

Trigger: Before insert/update on results recalculates grade and pass/fail status from marks.

8. User Interface

The interface provides a dashboard plus separate pages for student, department, course, enrollment and result management. Forms use parameterized SQL statements to reduce injection risk and to keep database operations structured.



Dashboard with aggregate counts and recent result records.

Student CRUD

SIMS DBMS Mini Project

Dashboard

Students

Departments

Courses

Enrollments

Results

Student Management

CRUD operations with constraints, prepared statements and foreign keys.

Add Student

Enrollment No.

IT2026010

Email

vansh@example.com

City

Nagpur

First Name

Vansh

Phone

9876500020

Admission Year

2026

Last Name

Rahate

Department

Information Technology

Status

Active

Create Student

Student Records

ID	Enrollment	Name	Email	Department	City	Status	Actions
8	CIVIL2025006	Ishita Raut	ishita.raut@example.com	Civil Engineering	Nagpur	Active	Edit Delete
7	ME2025003	Aditya Gawande	aditya.g@example.com	Mechanical Engineering	Akola	Active	Edit Delete
6	IT2024009	Meera Deshpande	meera.d@example.com	Information Technology	Nagpur	Graduated	Edit Delete

Student form and tabular CRUD view.

Enrollment CRUD

SIMS DBMS Mini Project

DashboardStudentsDepartmentsCoursesEnrollmentsResults

Enrollments

Many-to-many relationship between students and courses.

Create Enrollment

Student

IT2026001 - Aarav Patil

Course

IT401 - Database Management Systems

Semester

4

Academic Year

2026-27

Enrolled On

2026-07-05

Create Enrollme

ID	Student	Course	Semester	Academic Year	Date
14	Ishita Raut	CIVIL304	3	2026-27	2026-07-07
13	Aditya Gawande	MECH303	3	2026-27	2026-07-07
1	Aarav Patil	IT401	4	2026-27	2026-07-05
2	Aarav Patil	IT402	4	2026-27	2026-07-05

Bridge table: enrollments(student_id, course_id, semester_no, academic_year)

Enrollment screen showing the student-course relationship.

Result and Trigger UI

SIMS DBMS Mini Project

DashboardStudentsDepartmentsCoursesEnrollments**Results**

Results

Marks are entered through the UI; a trigger calculates grade and pass/fail status.

Save Result

Enrollment

Aarav Patil - IT401

Marks (0-100)

88

Evaluated On

2026-07-20

Remarks

Strong DBMS concepts

Save Result

ID	Student	Course	Marks	Grade	Status	Remarks
1	Aarav Patil	IT401	88	A	Pass	Strong DBMS concepts
4	Riya Sharma	IT401	91	A+	Pass	Excellent
5	Riya Sharma	IT402	72	B	Pass	Needs more practice
6	Kabir Joshi	CSE301	84	A	Pass	Good OS fundamentals

BEFORE INSERT/UPDATE trigger calls fn_grade(marks) and sets pass/fail status.

Result screen explaining that grade and pass/fail status are trigger-driven.

9. Conclusion

The Student Information Management System consolidates the DBMS laboratory syllabus into one practical database application. It demonstrates how relational design, normalization, constraints and SQL features work together to support real CRUD workflows. It also shows how views, stored routines, triggers and transactions can be used in an application rather than only as isolated lab statements.

10. Future Scope

Possible extensions include role-based application login, attendance management, fee records, document uploads, CSV/PDF report exports, REST APIs, pagination/search, audit dashboards and deployment on a cloud MySQL service.

Appendix A - Setup Summary

1. Run database/student_management.sql. 2. Copy public/config.example.php to public/config.php and update MySQL credentials. 3. Start PHP using php -S localhost:8000 from the public folder. 4. Run database/queries.sql to demonstrate the laboratory topics. 5. Review database/admin.sql for users/roles and database/backup_restore.md for backup/recovery.

Appendix B - MySQL Adaptation Note

The supplied manual contains Oracle-oriented syntax and PL/SQL examples. Because this project is required to use MySQL, the implementation uses MySQL stored procedures/functions in place of PL/SQL, MySQL account/role administration in place of Oracle-specific user examples, and a UNION-based full outer join emulation. The topic intent is preserved while the syntax is changed to match MySQL.